

Kubernetes – GitOps avec ArgoCD et Helm

Jour 5 — GitOps avec ArgoCD

Git comme source unique de vérité

Kubernetes — GitOps avec ArgoCD et Helm — m2i
Module 5/5 (Module 2 / Jour 2)

Objectifs du module

À la fin de cette journée, vous serez capable de :

- Expliquer les 4 principes du GitOps
- Installer ArgoCD et accéder à son UI et sa CLI
- Créer une Application ArgoCD qui consomme un chart Helm
- Mettre en place un dépôt multi-environnements avec App of Apps ou ApplicationSet
- Gérer un incident : OutOfSync, auto-heal, rollback Git

Durée : 7 heures (1 journée)

Plan

1. Principes GitOps
2. Rappels Git orientés GitOps
3. Architecture et installation ArgoCD
4. L'objet Application
5. ArgoCD et Helm
6. Stratégies multi-environnements
7. Exploitation au quotidien

1. Principes GitOps

Définition

“ "GitOps : système opéré dans lequel l'état désiré est versionné dans Git, et un agent réconcilie en continu l'état réel vers l'état désiré." ”

Origines : Weaveworks (2017). Convergence vers Kubernetes.

L'idée n'est pas neuve (infrastructure-as-code) ; ce qui change : la réconciliation pull, pilotée par un agent dans le cluster.

Les 4 principes

1. **Déclaratif** : l'état désiré est décrit, pas l'enchaînement d'actions
2. **Versionné et immuable** : Git est la source unique. Chaque état = un commit
3. **Approuvé automatiquement** : PR + CI = porte d'entrée. Sync auto sur main
4. **Réconcilié en continu** : un agent in-cluster détecte les drifts et corrige

À noter : *réconcilié en continu* $\neq$ *appliqué une fois*.

Push vs Pull

Push (CI/CD classique)

- Jenkins / GitHub Actions pousse `kubectl apply`
- Le pipeline a les credentials cluster
- Drift = invisible
- Audit fragmenté

Pull (GitOps)

- Un agent pull depuis Git
- Pas de credentials cluster côté CI
- Drift = détecté et réparé
- Audit unifié dans Git

Bénéfices

- **Traçabilité** : qui a déployé quoi, quand, pourquoi (= `git log`)
- **Auditabilité** : conformité ISO/SOC2 facilitée
- **Reproductibilité** : `git checkout <sha>` reproduit l'état du cluster
- **Sécurité** : pas de credentials cluster en CI
- **Disaster recovery** : reconstruire le cluster = sync depuis Git

Ces bénéfices sont structurels, pas accessoires.

Panorama : ArgoCD vs Flux

	ArgoCD	Flux
UI riche	Oui	Non (Weave Gitops séparé)
Multi-cluster natif	Oui	Oui
ApplicationSet	Oui	Kustomization + sources
Pattern App of Apps	Oui (canonique)	Possible
Adoption CNCF	Graduated	Graduated

Tous deux GA. Choix souvent par préférence UI et écosystème.

2. Rappels Git orientés GitOps

Trunk-based vs branches par environnement

Trunk-based

- Une seule branche `main`
- Promotion par sous-dossier (`environments/{dev,prod}`)
- Simple, traçable
- Recommandé pour ArgoCD

Branche par env

- `dev`, `staging`, `prod`
- Merge progressif
- Plus visuel mais complexe
- Risque de divergence

Monorepo vs multirepo

Critère	Monorepo	Multirepo
Visibilité	Une vue d'ensemble	Cloisonnement par équipe
Refactor cross-app	Facile	PR multiples
RBAC fin	Difficile	Naturel
Démarrage	Recommandé	Quand >50 services

Pour la formation : monorepo, c'est la voie pédagogique.

PR et revue : la porte d'entrée

```
développeur      CI      ArgoCD
|                |      |
|--- PR -----> lint+test  |
|                |      |
|<-- review ---|      |
|--- merge -----> deploy folder  |
|                |      |
|                |      |--- pull --> sync cluster
```

Le déploiement n'a pas de credentials sur le cluster. Tout passe par Git.

Séparation code / manifestes

Deux repos (recommandé) :

- `m2i/demo-api` : code Java, Dockerfile, pom.xml, CI build+push image
- `m2i/m2i-gitops` : charts Helm, values, Applications ArgoCD

Avantage : la CI applicative peut bumper le tag d'image dans le repo GitOps via PR auto, sans toucher au code.

3. Architecture et installation d'ArgoCD

Composants

```
+----- ArgoCD (namespace argocd) -----+
|
| argocd-server      <-- UI/API, RBAC
| argocd-repo-server <-- clone et rend les manifestes
| argocd-application-controller <-- la boucle de sync
| argocd-dex-server  <-- OIDC, optionnel
| argocd-redis       <-- cache
|
+-----+
|
| v repo: read-only
+-----+
| Git repo |
+-----+
```

Boucle de réconciliation

```
toutes les 3 minutes (configurable) :  
1. repo-server clone le repo  
2. repo-server rend les manifestes (helm template ou kustomize build)  
3. controller compare au cluster (live state)  
4. si différent : status = OutOfSync  
5. si syncPolicy.automated : applique
```

Et un webhook Git accélère la détection à <5s.

Installation

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/\
argoproj/argo-cd/v2.13.0/manifests/install.yaml

kubectl wait --namespace argocd \
  --for=condition=available deployment --all --timeout=300s
```

Pour un cluster Kind : pas besoin de high availability. En prod : `install-ha.yaml`.

Accès UI

```
kubectl -n argocd port-forward svc/argocd-server 8080:443  
# https://localhost:8080 (cert auto-signé)
```

Mot de passe initial :

```
kubectl -n argocd get secret argocd-initial-admin-secret \  
-o jsonpath="{.data.password}" | base64 -d
```

À changer dès la première connexion.

CLI argocd

```
brew install argocd      # macOS
# linux : curl + chmod

argocd login localhost:8080 \
  --username admin --password "$PWD" --insecure

argocd cluster list
argocd app list
argocd app get demo-api-dev
argocd app sync demo-api-dev
argocd app history demo-api-dev
```

4. L'objet Application

Anatomie

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: demo-api-dev
  namespace: argocd
spec:
  project: m2i
  source:
    repoURL: https://github.com/m2i/m2i-gitops
    targetRevision: main
    path: apps/demo-api-chart
    helm:
      releaseName: demo-api
      valueFiles:
        - values.yaml
        - ../../gitops/environments/dev/values.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: demo-api-dev
  syncPolicy:
    automated: { prune: true, selfHeal: true }
    syncOptions:
      - CreateNamespace=true
```

Sources supportées

Type	Détection
Helm chart	Présence d'un <code>Chart.yaml</code>
Kustomize	Présence d'un <code>kustomization.yaml</code>
Directory (YAML brut)	Dossier de manifestes
Plugin custom	Via <code>argocd-cmp-server</code>

ArgoCD ne lance pas `helm install`. Il appelle `helm template` puis applique le résultat.

Status d'une Application

Status	Signification
OutOfSync	État cluster $\neq$ état Git
Synced	OK
Healthy	Toutes les ressources sont saines (Pods Ready, etc.)
Degraded	Au moins une ressource KO
Progressing	Sync en cours
Missing	Pas encore appliquée

Une Application peut être **Synced** mais **Degraded** (le YAML est appliqué, mais l'app crashe).

Création : 3 voies

```
# 1. UI (clickodrome)

# 2. CLI
argocd app create demo-api-dev \
  --repo https://github.com/m2i/m2i-gitops \
  --path apps/demo-api-chart \
  --dest-server https://kubernetes.default.svc \
  --dest-namespace demo-api-dev \
  --sync-policy automated

# 3. YAML déclaratif (RECOMMANDÉ)
kubectl apply -f gitops/applications/demo-api-dev.yaml
```

La 3e voie est elle-même GitOps : ArgoCD se gère lui-même.

5. ArgoCD et Helm

ArgoCD ≠ `helm install`

“ ArgoCD n'utilise pas Helm pour installer. Il rend les manifestes avec `helm template`, puis applique. ”

Conséquences :

- Pas de `helm list` ni `helm rollback` côté cluster
- Pas de hooks Helm classiques (utiliser les sync waves ArgoCD)
- Le rollback se fait via Git (revert) ou via l'historique ArgoCD

Passer des values à un chart

```
spec:
  source:
    helm:
      releaseName: demo-api
      valueFiles:
        - values.yaml
        - ../../gitops/environments/prod/values.yaml
      values: |
        # values inline (priorité haute)
        ingress:
          enabled: true
      parameters:
        - name: replicaCount
          value: "5"
        - name: image.tag
          value: "1.2.3"
```

Ordre : `valueFiles` < `values` inline < `parameters`.

Sous-charts et dépendances

```
spec:  
  source:  
    helm:  
      passCredentials: false  
      ignoreMissingValueFiles: false  
      skipCrds: false
```

Pour les dépendances déclarées dans `Chart.yaml` : ArgoCD exécute `helm dependency update` automatiquement avant le template.

OCI privé

```
spec:
  source:
    repoURL: oci://registry.m2i.local/charts
    chart: demo-api
    targetRevision: 0.1.0
  helm:
    values: |
      replicaCount: 3
```

Repo OCI = pas de `path`, juste `chart`. Auth via Secret K8s ou repo configuration ArgoCD.

6. Multi-environnements

App of Apps

```
# applications/root.yaml – une seule Application racine
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: root
spec:
  source:
    repoURL: <REPO_URL>
    path: gitops/applications
    directory:
      recurse: false
      include: 'demo-api-*.yaml'
  syncPolicy:
    automated: { prune: true, selfHeal: true }
```

`root` synchronise toutes les `Application` du dossier. Pattern récursif.

ApplicationSet

```
kind: ApplicationSet
spec:
  generators:
    - git:
        repoURL: <REPO_URL>
        revision: main
        directories:
          - path: gitops/environments/*
  template:
    metadata:
      name: 'demo-api-{{path.basename}}'
    spec:
      source:
        path: apps/demo-api-chart
        helm:
          valueFiles:
            - ../../{{path}}/values.yaml
      destination:
        namespace: 'demo-api-{{path.basename}}'
```

Ajouter `gitops/environments/qa/values.yaml` → ArgoCD crée automatiquement `demo-api-qa`.

Générateurs ApplicationSet

Générateur	Cas d'usage
List	Liste explicite d'envs
Cluster	Une App par cluster enregistré
Git (directory)	Une App par sous-dossier
Git (file)	Une App par fichier (config par env dans JSON/YAML)
Matrix	Combinaison de générateurs (env × cluster)
SCM Provider	Une App par repo GitHub d'un org

Matrix est l'outil le plus puissant : `App = cluster × env`.

Projets ArgoCD

```
kind: AppProject
metadata: {name: m2i}
spec:
  sourceRepos:
    - 'https://github.com/m2i/*'
  destinations:
    - namespace: 'demo-api-*'
      server: https://kubernetes.default.svc
  clusterResourceWhitelist:
    - group: ''
      kind: Namespace
  roles:
    - name: developer
      policies:
        - 'p, proj:m2i:developer, applications, get, m2i/*, allow'
        - 'p, proj:m2i:developer, applications, sync, m2i/dev-*, allow'
```

Permet RBAC fin par équipe.

7. Exploitation au quotidien

Sync policies

```
syncPolicy:  
  automated:  
    prune: true      # supprime les ressources orphelines  
    selfHeal: true    # corrige les drifts dans le cluster  
    allowEmpty: false  
  syncOptions:  
    - CreateNamespace=true  
    - ServerSideApply=true  
    - PruneLast=true  
  retry:  
    limit: 5  
    backoff:  
      duration: 5s  
      factor: 2  
      maxDuration: 3m
```

`selfHeal: true` = un `kubectl edit` direct est réécrasé en moins de 5 minutes.

Sync waves

```
metadata:  
  annotations:  
    argocd.argoproj.io/sync-wave: "-1"    # avant les autres
```

Ordre par défaut : par kind (Namespace puis Secret puis ConfigMap puis Deployment).
Sync waves permettent d'imposer un ordre custom (DB avant API, par exemple).

Rollback : Git vs ArgoCD

Via Git (recommandé)

- `git revert <bad-sha>`
- PR + merge
- ArgoCD sync auto
- Audit complet

Via ArgoCD CLI/UI

- `argocd app rollback demo-api 12`
- Instantané
- Ne touche pas Git → drift à la prochaine sync
- À éviter en prod sauf incident

Health checks personnalisés

```
# Lua dans la configuration ArgoCD
data:
  resource.customizations.health.networking.istio.io_VirtualService: |
    hs = {}
    hs.status = "Healthy"
    return hs
```

ArgoCD a des checks built-in pour la plupart des ressources. Pour les CRDs maison, on écrit du Lua.

Notifications

```
kind: ConfigMap
metadata: {name: argocd-notifications-cm, namespace: argocd}
data:
  service.slack: |
    token: $slack-token
  trigger.on-sync-failed: |
    - when: app.status.operationState.phase in ['Error', 'Failed']
      send: [app-sync-failed]
  template.app-sync-failed: |
    message: "Sync failed for {{.app.metadata.name}}"
```

Slack, Teams, Webhook, Email, etc. Hors scope formation mais important en prod.

RBAC : projets et rôles

Rôle	Capacités
admin	Tout
developer	get, sync sur apps de son projet
viewer	get seul

```
g, m2i-developers, proj:m2i:developer
g, alice@m2i.fr, role:admin
```

Lié à OIDC (Keycloak, Google, GitHub) en prod.

Démo live

Démo 5 — Installer ArgoCD et créer une première Application

Voir `demos/demo-05.md`

Démo live

Démo 6 — App ArgoCD pointant vers le chart Helm demo-api

Voir `demos/demo-06.md`

Démo live

Démo 7 — Multi-env via ApplicationSet

Voir `demos/demo-07.md`

Démo live

Démo 8 — Auto-heal et rollback Git

Voir `demos/demo-08.md`

TP à suivre

TP 6 — Installer ArgoCD sur Kind · 30 min · [tp/tp-06.md](#)

TP 7 — Structurer le dépôt GitOps · 30 min · [tp/tp-07.md](#)

TP 8 — Première Application sur dev · 45 min · [tp/tp-08.md](#)

TP 9 — Stack métier complète · 90 min · [tp/tp-09.md](#)

TP à suivre (suite)

TP 10 — ApplicationSet multi-env · 45 min · `tp/tp-10.md`

TP 11 — Incident, auto-heal et rollback · 45 min · `tp/tp-11.md`

TP 12 — Synthèse collective · 30 min · `tp/tp-12.md`

Récap du module

- GitOps = déclaratif + versionné + approuvé + réconcilié en continu
- ArgoCD = controller in-cluster qui pull et applique
- Application = ce qu'on déploie, ApplicationSet = comment on les multiplie
- Helm est rendu côté ArgoCD, pas installé
- Rollback Git > rollback ArgoCD (sauf incident)
- Self-heal réécrit les `kubectl edit` directs

Cheatsheet : `cheatsheet/module-05.md`

Questions

Fin du parcours Kubernetes 2 modules — m2i

